

1. Explain Functional Dependency and its Types with suitable examples.

A Functional Dependency (FD) in a Relational Database Management System is a relationship between two sets of attributes where the value of one attribute determines the value of another attribute.

It is denoted by: $X \rightarrow Y$

Where:

- X = Determinant
- Y = Dependent attribute

Functional Dependencies are used to mathematically express relationships among database entities and play a vital role in database normalization.

Example

Consider the following STUDENT table:

Roll_No	Name	Dept_Name	Dept_Building
42	ABC	CO	A4
43	PQR	IT	A3
44	XYZ	CO	A4
45	XYZ	IT	A3
46	MNO	EC	B2
47	JKL	ME	B2

Valid Functional Dependencies

- Roll_No $\rightarrow$ Name
- Roll_No $\rightarrow$ Dept_Name
- Roll_No $\rightarrow$ Dept_Building
- Roll_No $\rightarrow$ {Name, Dept_Name, Dept_Building}
- Dept_Name $\rightarrow$ Dept_Building

Invalid Functional Dependencies

- Name $\rightarrow$ Dept_Name
- Dept_Building $\rightarrow$ Dept_Name

because the same name/building may correspond to multiple departments.

Types of Functional Dependencies

1. Trivial Functional Dependency

A Functional Dependency is called Trivial when the dependent attribute is a subset of the determinant.

Condition:

If $X \rightarrow Y$ and $Y \subseteq X$

Example

$\{\text{Roll_No, Name}\} \rightarrow \text{Name}$

or

$\text{Roll_No} \rightarrow \text{Roll_No}$

Here, the dependent attribute already exists in the determinant.

2. Non-Trivial Functional Dependency

A Functional Dependency is called Non-Trivial when the dependent attribute is not a subset of the determinant.

Condition

If $X \rightarrow Y$ and $Y \not\subseteq X$

Example

$\text{Roll_No} \rightarrow \text{Name}$

$\text{Roll_No} \rightarrow \text{Age}$

Here, Name and Age are not part of Roll_No. Hence, it is a Non-Trivial Functional Dependency.

3. Multivalued Functional Dependency

A Multivalued Functional Dependency exists when one attribute determines multiple independent attributes.

There is no dependency between the dependent attributes themselves.

Example

Roll_No	Name	Age
42	ABC	17
43	PQR	18
44	XYZ	18
45	ABC	19

$\text{Roll_No} \rightarrow \{\text{Name}, \text{Age}\}$

Since Name does not determine Age and Age does not determine Name, this is a Multivalued Functional Dependency.

4. Transitive Functional Dependency

A dependency is Transitive when one attribute depends on another indirectly.

If $A \rightarrow B$ and $B \rightarrow C$ then $A \rightarrow C$

Example

$\text{Enroll_No} \rightarrow \text{Dept}$

$\text{Dept} \rightarrow \text{Building_No}$

Therefore,

$\text{Enroll_No} \rightarrow \text{Building_No}$

This is called a Transitive Functional Dependency.

5. Fully Functional Dependency

A Functional Dependency is Full Functional Dependency when the entire key determines another attribute.

Example

$\text{Roll_No} \rightarrow \text{Name}$

$\text{Roll_No} \rightarrow \text{Age}$

Here, Roll_No alone uniquely determines Name and Age.

6. Partial Functional Dependency

A Partial Functional Dependency occurs when a non-key attribute depends only on part of a composite key, rather than on the whole composite key.

Example

Suppose the primary key is:

(Student_ID, Course_ID)

If

Course_ID → Course_Fee

instead of

(Student_ID, Course_ID) → Course_Fee

then Course_Fee depends only on part of the composite key.

Hence, it is called Partial Functional Dependency.

Advantages of Functional Dependency

Functional Dependencies provide several benefits:

1. Data Normalization

- Helps organize data.
- Reduces redundancy.
- Improves data integrity.

2. Query Optimization

- Identifies relationships between tables.
- Improves query performance.

3. Data Consistency

- Eliminates redundancy.
- Ensures consistent data during updates.

4. Data Quality Improvement

- Improves accuracy, completeness, and reliability of stored data.

Functional Dependency: X -----> Y

Determinant Dependent

2.Explain Armstrong's Axioms (Properties of Functional Dependencies) with suitable examples.

Armstrong's Axioms are a set of inference rules used to derive new Functional Dependencies (FDs) from the existing Functional Dependencies.

These axioms help to:

- Find all possible functional dependencies.
- Check dependency preservation.
- Design and normalize relational databases efficiently.

Armstrong's Axioms

There are three basic axioms:

1. Reflexivity Rule
2. Augmentation Rule
3. Transitivity Rule

1. Reflexivity Rule:

If Y is a subset of X, then $X \rightarrow Y$.

Rule:

If $Y \subseteq X$, then $X \rightarrow Y$

Example:

$\{\text{Roll_No, Name}\} \rightarrow \text{Name}$

Since Name is already a part of {Roll_No, Name}, the dependency is always true.

2. Augmentation Rule:

If $X \rightarrow Y$ is true, then adding the same attribute(s) to both sides also results in a valid Functional Dependency.

Rule:

If $X \rightarrow Y$, then $XZ \rightarrow YZ$

Example:

Given:

$\{\text{Roll_No, Name}\} \rightarrow \text{Dept_Building}$

By adding Dept_Name to both sides,

$\{\text{Roll_No}, \text{Name}, \text{Dept_Name}\} \rightarrow \{\text{Dept_Building}, \text{Dept_Name}\}$

Hence, the dependency remains valid.

3. Transitivity Rule:

If one attribute determines a second attribute, and the second attribute determines a third attribute, then the first attribute determines the third attribute.

Rule:

If $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$

Example:

$\text{Roll_No} \rightarrow \text{Dept_Name}$

$\text{Dept_Name} \rightarrow \text{Dept_Building}$

Therefore,

$\text{Roll_No} \rightarrow \text{Dept_Building}$

This is called Transitive Functional Dependency.

Summary Table

Armstrong's Axiom	Rule	Example
Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$	$\{\text{Roll_No}, \text{Name}\} \rightarrow \text{Name}$
Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$	$\{\text{Roll_No}, \text{Name}\} \rightarrow \text{Dept_Building} \Rightarrow \{\text{Roll_No}, \text{Name}, \text{Dept_Name}\} \rightarrow \{\text{Dept_Building}, \text{Dept_Name}\}$
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$	$\text{Roll_No} \rightarrow \text{Dept_Name}, \text{Dept_Name} \rightarrow \text{Dept_Building} \Rightarrow \text{Roll_No} \rightarrow \text{Dept_Building}$

Diagram to Draw Armstrong's Axioms

1. Reflexivity- $\{A,B\} \rightarrow A$

2. Augmentation- $A \rightarrow B$



3. Transitivity-A -----> B -----> C



Applications of Armstrong's Axioms:

- Derive new Functional Dependencies.
- Find the closure of attributes.
- Verify dependency preservation.
- Assist in database normalization.
- Reduce data redundancy and improve database design.

3.Explain Normalization. Discuss 1NF, 2NF and 3NF with suitable examples.

Normalization is the process of organizing data in a database to minimize redundancy and eliminate undesirable anomalies such as insertion, update, and deletion anomalies.

It divides a large table into smaller related tables and links them using relationships.

Objectives of Normalization:

- Minimize data redundancy.
- Improve data integrity.
- Eliminate insertion, update and deletion anomalies.
- Improve database consistency.

Normalization is required to:

- Reduce duplicate data.
- Save storage space.
- Maintain consistency.
- Simplify database maintenance.
- Prevent anomalies.
- Improve database efficiency.

Normal Forms

Normalization is performed through different stages called Normal Forms (NF).

They are:

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)

1. First Normal Form (1NF)

A relation is in First Normal Form (1NF) if every attribute contains only atomic (single) values.

It does not allow:

- Multi-valued attributes
- Composite attributes
- Multiple values in one cell

Example (Not in 1NF)

Emp_ID	Name	Phone_No
101	Rahul	98765, 91234
102	Ravi	99887

Here, Phone_No contains multiple values.

Converted to 1NF

Emp_ID	Name	Phone_No
101	Rahul	98765
101	Rahul	91234
102	Ravi	99887

Now every cell contains only one value.

Hence, the relation is in 1NF.

2. Second Normal Form (2NF)

A relation is in Second Normal Form (2NF) if:

- It is already in 1NF.
- It has No Partial Dependency.

A non-prime attribute should depend on the entire candidate key, not on a part of it.

Example

STUD_NO	COURSE_NO	COURSE_FEE
1	C1	1000
2	C2	1500
1	C4	2000
4	C3	1000
4	C1	1000
2	C5	2000

Primary Key:

(STUD_NO, COURSE_NO)

But,

COURSE_NO → COURSE_FEE

Here, COURSE_FEE depends only on COURSE_NO, which is only a part of the composite key.

Hence, Partial Dependency exists

Therefore, the table is NOT in 2NF.

Conversion to 2NF

Table-1

STUD_NO	COURSE_NO
1	C1
2	C2
1	C4
4	C3
4	C1
2	C5

Table-2

COURSE_NO	COURSE_FEE
C1	1000
C2	1500
C3	1000
C4	2000
C5	2000

Now there is No Partial Dependency.

Hence, it is in 2NF.

3. Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if:

- It is already in 2NF.
- It has No Transitive Dependency.

A non-key attribute should depend only on the primary key.

Conditions for 3NF:

For every Functional Dependency

$X \rightarrow Y$

Either,

- X is a Super Key

OR

- Y is a Prime Attribute

Example:

Suppose, $(\text{STUD_NO}, \text{COURSE_NO}) \rightarrow \text{COURSE_FEE}$

But actually, $\text{COURSE_NO} \rightarrow \text{COURSE_FEE}$

Therefore,

$(\text{STUD_NO}, \text{COURSE_NO})$

↓

COURSE_NO

↓

COURSE_FEE

This is a Transitive Dependency.

Hence, the relation is NOT in 3NF.

After removing the transitive dependency, the relation satisfies 3NF.

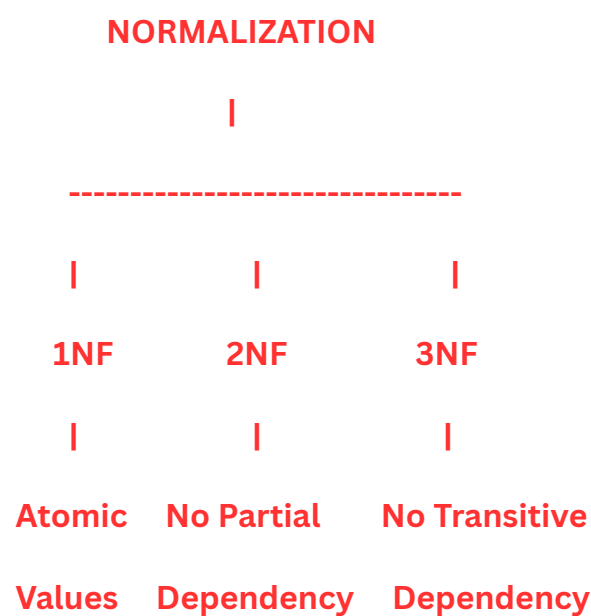
Comparison of 1NF, 2NF and 3NF

Normal Form	Condition	Removes
1NF	Atomic values only	Multi-valued attributes
2NF	1NF + No Partial Dependency	Partial Dependency
3NF	2NF + No Transitive Dependency	Transitive Dependency

Advantages of Normalization

- Reduces data redundancy.
- Eliminates insertion anomalies.
- Eliminates update anomalies.
- Eliminates deletion anomalies.
- Improves consistency.
- Improves data integrity.
- Simplifies database maintenance.
- Efficient utilization of storage.

Diagram:



4.Explain BCNF, 4NF and 5NF with suitable examples.

After achieving Third Normal Form (3NF), some redundancy and anomalies may still remain. To further eliminate these problems, higher normal forms are used:

- Boyce-Codd Normal Form (BCNF)
- Fourth Normal Form (4NF)
- Fifth Normal Form (5NF)

These normal forms improve database design by reducing redundancy and ensuring better data integrity.

1. Boyce-Codd Normal Form (BCNF)

BCNF (Boyce-Codd Normal Form) is an advanced version of Third Normal Form (3NF).

A relation is in BCNF if:

- It is already in 3NF.
- For every non-trivial functional dependency ($X \rightarrow Y$), X must be a Super Key.

Rules of BCNF

1. The table must be in Third Normal Form.
2. Every determinant should be a Super Key.
3. To check BCNF, identify all determinants and verify that each is a candidate/super key.

Example

Consider:

Student_ID	Course	Instructor
101	DBMS	Ramesh
102	OS	Suresh

Functional Dependency:

Course → Instructor

Here,

- Course is not a Super Key.

Therefore,

The relation is NOT in BCNF.

After decomposition so that every determinant becomes a super key, the relation satisfies BCNF.

2. Fourth Normal Form (4NF)

A relation is in Fourth Normal Form (4NF) if:

- It is already in BCNF.
- It contains no non-trivial Multivalued Dependency.

Rules of 4NF

1. Relation must be in BCNF.
2. It should not contain Multivalued Dependency.

Multivalued Dependency

A multivalued dependency occurs when:

For a single value of one attribute, there are multiple independent values of another attribute.

Notation:

A → B

Example

Student	Hobby	Language
Rahul	Cricket	English
Rahul	Music	English
Rahul	Cricket	Hindi
Rahul	Music	Hindi

Here,

- One student has multiple hobbies.
- One student has multiple languages.
- Hobby and Language are independent.

Therefore,

Student → Hobby

Student → Language

This relation has Multivalued Dependency and is not in 4NF.

It is converted into 4NF by decomposing it into separate tables.

3. Fifth Normal Form (5NF)

Fifth Normal Form (5NF) is also called Projected Normal Form (PJNF).

A relation is in 5NF if:

- It is already in 4NF.
- It cannot be further decomposed without losing information.
- Every Join Dependency is implied by the Candidate Keys.

Rules of 5NF

1. Relation must be in 4NF.

2. It should have Lossless Join Property.
3. It should not have unnecessary Join Dependency.

Example

Suppose we have a table:

Supplier	Product	Customer
Ali	ABC	Nauman
Ali	XYZ	Ahmed

The relation is decomposed into:

- Supplier – Product
- Product – Customer
- Supplier – Customer

After decomposition,

- No information is lost.
- Original table can be reconstructed using Natural Join.

Hence, the relation is in 5NF.

Comparison of BCNF, 4NF and 5NF

Normal Form	Condition	Removes
BCNF	Every determinant must be a Super Key	Functional dependency anomalies
4NF	BCNF + No Multivalued Dependency	Multivalued Dependency
5NF	4NF + No Join Dependency	Join Dependency

Advantages

- Eliminates redundancy.
- Improves consistency.
- Prevents update anomalies.
- Produces better database design.
- Ensures lossless decomposition.

- Improves data integrity.

Diagram to Draw



5.Explain Domain-Key Normal Form (DKNF). Is it practical?

Domain-Key Normal Form (DKNF) is the highest level of database normalization.

A relation is said to be in DKNF if all constraints in the relation are a logical consequence of only Domain Constraints and Key Constraints.

In other words, a database is in DKNF when there are no constraints other than domain constraints and key constraints.

Domain Constraint

A Domain Constraint specifies the valid set of values that an attribute can take.

Examples

- Age must be between 18 and 60.
- Salary must be greater than 0.
- Gender can be Male, Female, or Other.

Thus, a domain defines the permitted values of an attribute.

Key Constraint:

A Key Constraint ensures that every record in a table is uniquely identified.

Examples of keys include:

- Primary Key
- Candidate Key
- Foreign Key

The key constraint guarantees that no two rows have the same primary key value.

Conditions for DKNF

A relation is in DKNF if:

1. Every constraint is a result of Domain Constraints.
2. Every constraint is a result of Key Constraints.
3. No additional constraints are required.

Thus, the database becomes free from all types of anomalies.

Steps to Achieve DKNF:

1. Eliminate repeating groups.
2. Remove repeated (redundant) data.
3. Eliminate columns that are not dependent on the key.
4. Separate multiple relationships independently

Advantages of DKNF

- Eliminates data redundancy.
- Removes insertion, deletion, and update anomalies.
- Improves data consistency.
- Improves data integrity.
- Produces a well-structured database.
- Ensures efficient organization of data.

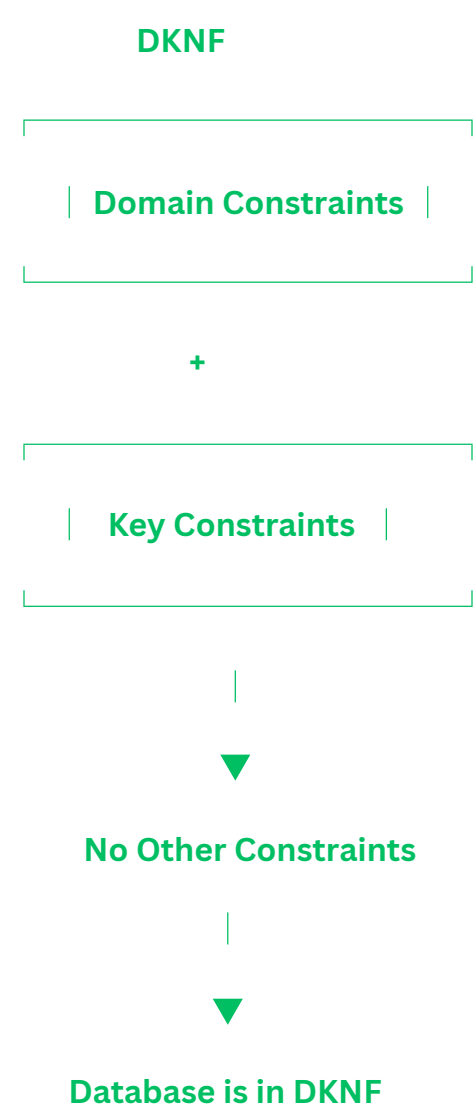
Is DKNF Practical?

DKNF is theoretically the best normal form, but it is not practical in most real-world database applications.

Reasons

- It is very difficult to design a database directly in DKNF.
- It requires careful identification of all domain and key constraints.
- It is challenging even for experienced database designers.
- Achieving DKNF increases database design complexity.
- Therefore, most real-world databases stop at 3NF or BCNF, as they provide a good balance between normalization and performance.

Diagram:



6.Explain Insertion, Update and Deletion Anomalies. How does Normalization remove these anomalies?

An Anomaly is an undesirable problem that occurs in a database due to data redundancy.

When the same data is stored repeatedly, operations such as Insertion, Update, and Deletion may lead to inconsistencies and data loss.

Normalization removes these anomalies by organizing data into well-structured tables.

Types of Anomalies:

There are three types of anomalies:

1. **Insertion Anomaly**
2. **Update Anomaly**
3. **Deletion Anomaly**

1. Insertion Anomaly-

An Insertion Anomaly occurs when some information cannot be inserted into the database unless other related information already exists.

In other words, inserting a new record becomes difficult because another attribute is mandatory.

Example

Consider the STUDENT table:

Student_ID	Student_Name	Course_ID	Course_Name
101	Rahul	C1	DBMS
102	Ravi	C2	OS

Suppose a new course C3 (Python) is introduced, but no student has enrolled in it yet.

Since there is no Student_ID, the course details cannot be inserted into the table.

This is called an Insertion Anomaly.

2. Update Anomaly-

An Update Anomaly occurs when duplicate data is updated in one place but not in all places, resulting in inconsistent data.

Example

Student_ID	Student_Name	Course_Name
101	Rahul	DBMS
102	Ravi	DBMS

Suppose the course name DBMS is changed to Advanced DBMS.

If only the first record is updated:

Student_ID	Student_Name	Course_Name
101	Rahul	Advanced DBMS
102	Ravi	DBMS

Now, the database contains two different names for the same course, leading to inconsistency.

This is called an Update Anomaly.

3. Deletion Anomaly-

A Deletion Anomaly occurs when deleting one record accidentally removes other important information from the database.

Example

Student_ID	Student_Name	Course_Name
101	Rahul	DBMS
102	Ravi	OS

Suppose Rahul's record is deleted.

If Rahul was the only student enrolled in DBMS, deleting his record also removes all information about the DBMS course.

Thus, valuable information is lost.

This is called a Deletion Anomaly.

How Does Normalization Remove Anomalies?

Normalization divides a large table into smaller related tables, reducing redundancy and improving consistency.

Example

Instead of storing everything in one table:

STUDENT

Student_ID	Student_Name	Course_ID
101	Rahul	C1
102	Ravi	C2

COURSE

Course_ID	Course_Name
C1	DBMS
C2	OS

Now:

- New courses can be inserted independently (Insertion Anomaly removed).
- Course details need to be updated only once (Update Anomaly removed).
- Deleting a student does not remove course information (Deletion Anomaly removed).

Thus, normalization eliminates anomalies by reducing redundancy.

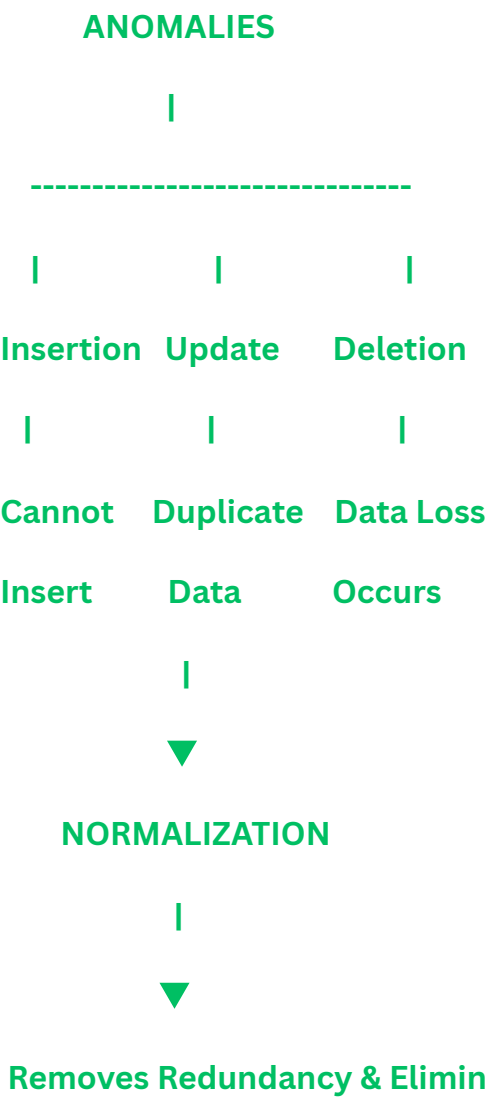
Advantages of Removing Anomalies

- Reduces data redundancy.
- Improves data consistency.
- Prevents accidental data loss.
- Maintains data integrity.
- Simplifies database maintenance.
- Improves database efficiency.

Summary Table

Anomaly	Meaning	Solution
Insertion Anomaly	Unable to insert data without other related data	Normalize the table
Update Anomaly	Updating duplicate data causes inconsistency	Store data only once using normalization
Deletion Anomaly	Deleting one record removes other useful information	Separate data into related tables

Diagram:



7.Explain Integrity Constraints and Enforcing Integrity Constraints.

Integrity Constraints are rules that ensure the validity, accuracy, and consistency of data stored in a database.

They prevent unauthorized or incorrect data from being inserted, updated, or deleted, thereby maintaining database integrity.

Need for Integrity Constraints

Integrity Constraints are used to:

- Maintain data accuracy.
- Ensure data consistency.
- Prevent invalid data entry.
- Avoid data redundancy.
- Protect the database from accidental modifications.

Types of Integrity Constraints:

There are four types of Integrity Constraints:

1. Domain Integrity
2. Entity Integrity
3. Referential Integrity
4. Key Constraints

1. Domain Integrity:

Domain Integrity ensures that an attribute contains only valid values.

A domain specifies:

- Data type
- Length/Size
- Range of values
- Default value
- NULL or NOT NULL
- Unique values

Example-

Age INT CHECK (Age >=18 AND Age<=60)

Only values between 18 and 60 are allowed.

Hence, Domain Integrity is maintained.

2. Entity Integrity Constraint:

The Primary Key of a relation cannot contain NULL values.

Every row must have a unique and valid primary key.

Example-

STUDENT

Student_ID (PK)	Name
101	Rahul
102	Ravi

Valid ✓

Student_ID	Name
NULL	Rahul

Invalid ✗ because the Primary Key cannot be NULL.

3. Referential Integrity Constraint:

A Foreign Key must either:

- Match an existing Primary Key value in the referenced table, or
- Be NULL.

This maintains consistency between related tables.

Example-

DEPARTMENT

Dept_ID (PK)	Dept_Name
10	CSE
20	ECE

STUDENT

Student_ID	Dept_ID (FK)
101	10
102	20

Valid ✓

If a student has:

Dept_ID = 50

and 50 does not exist in the DEPARTMENT table,

then it violates Referential Integrity.

4. Key Constraints:

A Key Constraint ensures that a key uniquely identifies every tuple in a relation.

It includes:

- Primary Key
- Candidate Key
- Super Key
- Foreign Key

Example-

Roll_No	Name
101	Rahul
102	Ravi

Here,

Roll_No uniquely identifies every student.

Hence,

Roll_No satisfies the Key Constraint.

Enforcing Integrity Constraints

Integrity Constraints are enforced using various mechanisms provided by DBMS.

1. NOT NULL Constraint-

Ensures that a column cannot contain NULL values.

Example:

Student_Name VARCHAR(30) NOT NULL

2. CHECK Constraint-

Restricts values based on a condition.

Example:

CHECK (Age >=18)

3. UNIQUE Constraint-

Ensures that duplicate values are not allowed.

Example:

Email UNIQUE

4. PRIMARY KEY Constraint-

Ensures:

- Unique values
- No NULL values

Example:

Student_ID PRIMARY KEY

5. FOREIGN KEY Constraint-

Maintains Referential Integrity between two tables.

Example:

Dept_ID REFERENCES Department(Dept_ID)

These mechanisms ensure the correctness and completeness of data in the database.

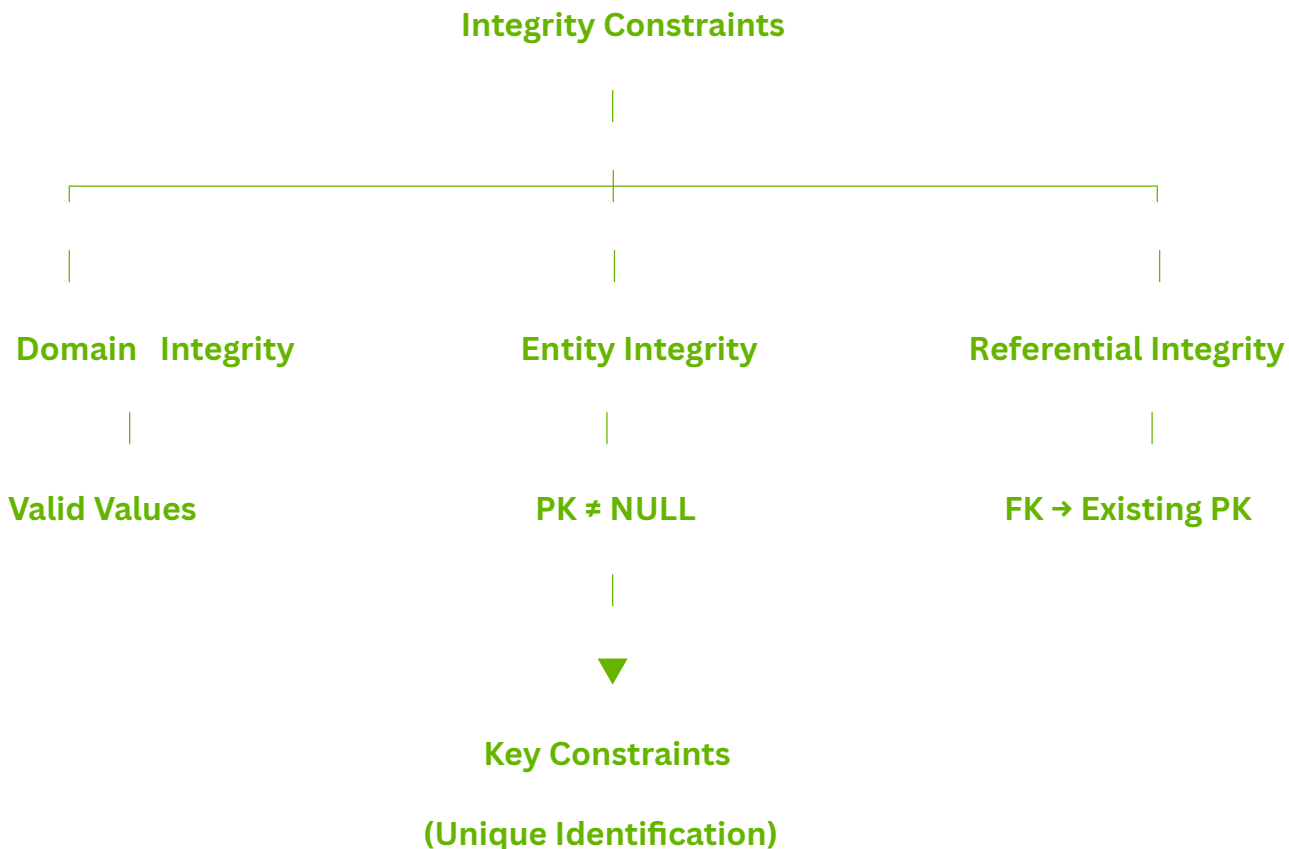
Summary Table:

Integrity Constraint	Purpose
Domain Integrity	Allows only valid values
Entity Integrity	Primary Key cannot be NULL
Referential Integrity	Foreign Key must match Primary Key
Key Constraint	Ensures uniqueness of records

Advantages of Integrity Constraints

- Ensures data accuracy.
- Maintains consistency.
- Prevents invalid data entry.
- Maintains relationships between tables.
- Improves database reliability.
- Prevents accidental errors.

Diagram:



8.Explain Views (Creation, WITH CHECK OPTION, Updating, Altering and Dropping Views)

A View is a virtual table created from one or more database tables using an SQL query.

It does not store data physically; instead, it stores the SQL query used to generate the data whenever the view is accessed.

Features of a View-

- It is a virtual table.
- It is created using the CREATE VIEW statement.
- It can be created from one or multiple tables.
- It stores only the SQL query, not the actual data.
- Changes in the base table are reflected in the view.

Advantages of Views-

Views allow users to:

- Present data in a simple and meaningful way.
- Restrict access to confidential data.
- Display only required columns.
- Generate reports easily.
- Simplify complex SQL queries.

Creating a View-

A view is created using the CREATE VIEW statement.

Syntax:

CREATE VIEW view_name AS SELECT column1, column2, ... FROM table_name WHERE condition;

Example:

Suppose we have a table:

CUSTOMERS

Cust_ID	Name	Age	Address	Mobile
101	Rahul	22	Hyderabad	9876543210
102	Ravi	24	Warangal	9123456780

Create a view:

CREATE VIEW CUSTOMERS_VIEW AS SELECT Name, Age FROM CUSTOMERS;

Now the view contains only:

Name	Age
Rahul	22
Ravi	24

Displaying a View:

To display the records in a view:

SELECT * FROM CUSTOMERS_VIEW;

This retrieves the data stored in the virtual table.

Views WITH CHECK OPTION:

The WITH CHECK OPTION ensures that every INSERT and UPDATE performed through the view satisfies the condition specified in the view definition.

If the condition is violated, the operation is rejected.

Syntax:

```
CREATE VIEW CUSTOMERS_VIEW AS SELECT Name, Age FROM CUSTOMERS WHERE Age IS NOT NULL WITH CHECK OPTION;
```

Explanation-

Here,

- Only customers with Age NOT NULL appear in the view.
- Any INSERT or UPDATE through the view must also satisfy this condition.
- If Age = NULL, the operation is not allowed.

Updating a View:

A view can be updated only if it satisfies certain conditions.

Conditions for Updating a View

A view cannot contain:

- DISTINCT
- Summary (Aggregate) Functions
- Set Functions
- Set Operators
- ORDER BY clause
- Multiple tables
- Subqueries
- GROUP BY clause
- HAVING clause
- Calculated columns

Also,

- All NOT NULL columns from the base table must be included in the view for INSERT operations.

Example-

```
UPDATE CUSTOMERS_VIEW SET AGE = 35 WHERE Name = 'Ramesh';
```

This updates the base table, and the changes are automatically reflected in the view.

Altering a View:

An existing view can be modified using the ALTER VIEW statement.

Syntax:

```
ALTER VIEW view_name AS SELECT column1, column2 FROM table_name;
```

Purpose:

- Add or remove columns.
- Modify the SELECT query.
- Change the view definition.

Dropping a View:

A view is deleted using the DROP VIEW statement.

Syntax:

```
DROP VIEW view_name;
```

After executing this command:

- The view is permanently removed.
- The base table remains unchanged.

Summary Table-

Operation	SQL Command	Purpose
Create View	CREATE VIEW	Creates a new view
Display View	SELECT * FROM view_name	Displays records
Update View	UPDATE	Modifies data through the view
Alter View	ALTER VIEW	Changes the view definition
Drop View	DROP VIEW	Deletes the view

Diagram:



CREATE VIEW

CUSTOMERS_VIEW

Name | Age

SELECT / UPDATE

Advantages of Views:

- Improves security by hiding sensitive data.
- Simplifies complex queries.
- Provides logical data independence.
- Displays only required information.
- Makes report generation easier.
- Reflects updates made to the base table.

9.Explain Keys in RDBMS with suitable examples.

A Key is an attribute or a set of attributes that is used to identify each record uniquely in a relation (table).

Keys help in:

- Uniquely identifying records.
- Establishing relationships between tables.
- Maintaining data integrity.
- Avoiding duplicate records.

Types of Keys in RDBMS:

1. Super Key

2. **Candidate Key**
3. **Primary Key**
4. **Alternate Key**
5. **Unique Key**
6. **Composite Key**
7. **Foreign Key**
8. **Surrogate Key**

Example Table-

Consider the following EMPLOYEE table:

ID	SSN	Name	Phone	Email
101	A101	Rahul	9876543210	rahul@gmail.com
102	A102	Ravi	9876543211	ravi@gmail.com

Using this table, different keys are explained below.

1. Super Key:

A Super Key is a set of one or more attributes that can uniquely identify a tuple (record).

- It may contain extra (redundant) attributes.
- It may contain NULL values (except when chosen as Primary Key).

Examples:

{ID}

{SSN}

{ID, Name}

{ID, SSN}

{ID, Phone}

{ID, Email}

All these can uniquely identify a record.

2. Candidate Key:

A Candidate Key is a minimal Super Key.

It contains no redundant attributes.

Examples:

{ID}

{SSN}

{Email}

{Name, Phone}

These uniquely identify a record with no unnecessary attributes.

3. Primary Key:

A Primary Key is the Candidate Key selected to uniquely identify every record in a table.

Characteristics-

- Unique
- Cannot contain NULL values
- Only one Primary Key per table

Example:

Primary Key = {ID}

Here, every employee has a unique ID.

4. Alternate Key:

An Alternate Key is a Candidate Key that is not selected as the Primary Key.

Example:

If Primary Key = {ID} Then,

Alternate Keys:

{SSN}

{Email}

{Name, Phone}

5. Unique Key:

A Unique Key ensures that all values in an attribute are unique.

Unlike the Primary Key, it can contain NULL values.

Example:

Email UNIQUE

Each employee has a different email address.

6. Composite Key:

A Composite Key consists of more than one attribute that together uniquely identify a record.

Example:

{Name, Phone}

Neither Name nor Phone alone identifies a record uniquely, but together they do.

7. Foreign Key:

A Foreign Key is an attribute in one table that refers to the Primary Key of another table.

It is used to establish relationships between tables and maintain Referential Integrity.

Example-

DEPARTMENT

Dept_ID	Dept_Name
10	CSE
20	ECE

STUDENT

Student_ID	Name	Dept_ID
101	Rahul	10
102	Ravi	20

Here,

- Dept_ID in the STUDENT table is a Foreign Key.
- It references Dept_ID in the DEPARTMENT table.

8. Surrogate Key:

A Surrogate Key is a system-generated unique identifier.

- It is internally generated.
- It is not derived from application data.
- It is generally invisible to users.

Example

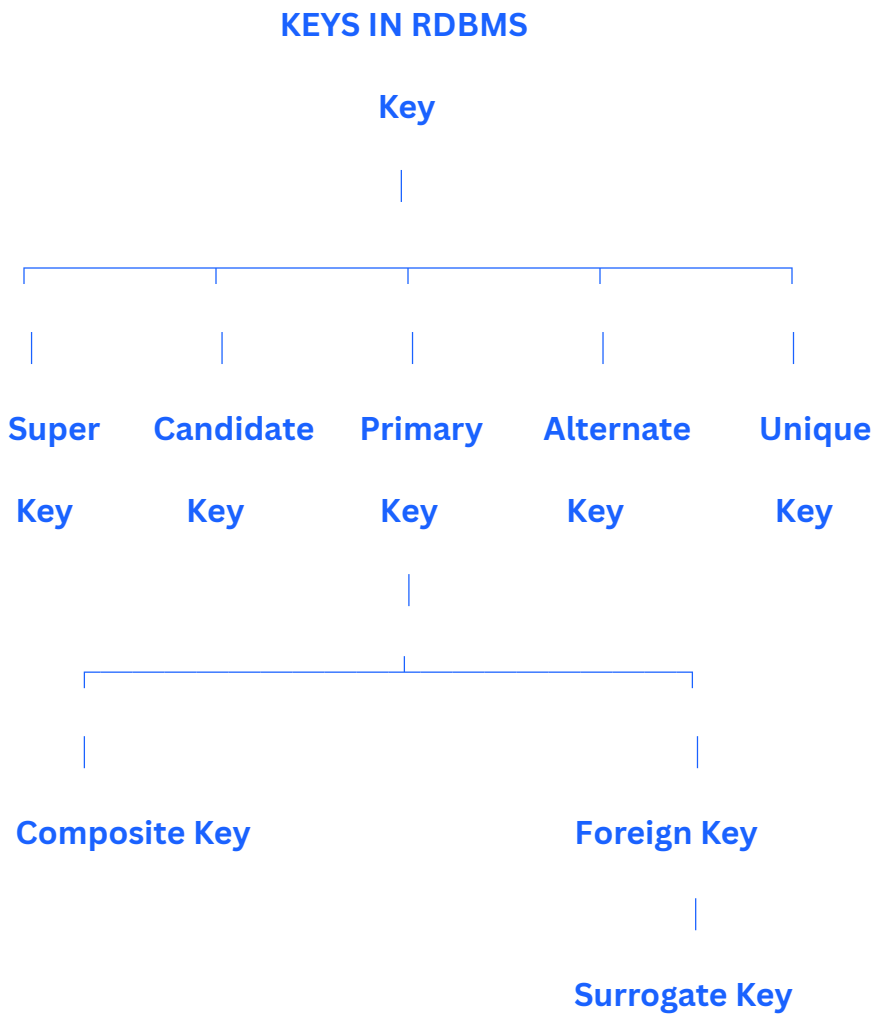
Key	Product_ID	Price
505_92	1987	200
698_56	1256	150
310_25	1875	225

Here, Key is the Surrogate Key.

Comparison of Keys

Key	Purpose
Super Key	Uniquely identifies records (may contain extra attributes)
Candidate Key	Minimal Super Key
Primary Key	Selected Candidate Key (Unique & Not NULL)
Alternate Key	Candidate Key not chosen as Primary Key
Unique Key	Ensures unique values (NULL allowed)
Composite Key	Combination of multiple attributes
Foreign Key	Links two tables
Surrogate Key	System-generated unique identifier

Diagram:



Advantages of Keys

- Uniquely identify records.
- Prevent duplicate entries.
- Maintain data integrity.
- Establish relationships between tables.
- Improve data retrieval efficiency.
- Support normalization.

10.Explain Logical Database Design (Database Schema, Relation Schema, Relation Instance and Schema Diagram).

Logical Database Design is the process of designing the logical structure of a database. It defines how data is organized into tables (relations), the attributes of those tables, and the relationships among them.

It focuses on the logical representation of data rather than its physical storage.

Components of Logical Database Design-

The main components are:

1. Database Schema
2. Database Instance
3. Relation Schema
4. Relation Instance
5. Schema Diagram

1. Database Schema:

A Database Schema is the logical design (blueprint) of the database.

It defines:

- Tables
- Attributes (Columns)
- Relationships
- Constraints

The schema generally does not change frequently.

Example:

STUDENT(Student_ID, Name, Branch)

DEPARTMENT(Dept_ID, Dept_Name)

This represents the database schema.

2. Database Instance:

A Database Instance is the actual data stored in the database at a particular point in time.

It is also called a snapshot of the database.

Unlike the schema, the instance changes whenever records are inserted, updated, or deleted.

Example:

STUDENT

Student_ID	Name	Branch
101	Rahul	CSE
102	Ravi	ECE

These records form the current database instance.

3. Relation Schema:

A Relation Schema defines the structure of a single relation (table).

It consists of:

- Relation name
- List of attributes
- Domains of attributes

Example-

DEPARTMENT(Dept, Campus, No_of_Rooms)

This specifies the structure of the DEPARTMENT relation.

4. Relation Instance:

A Relation Instance is the set of tuples (rows) present in a relation at a given time.

The contents of a relation instance change whenever the relation is updated.

Example

Dept	Campus	No_of_Rooms
CSE	Main	20
ECE	North	15

These rows represent the current relation instance.

Difference between Schema and Instance

Schema	Instance
Logical structure of the database	Actual data stored in the database
Changes rarely	Changes frequently
Also called blueprint	Also called snapshot
Defines tables and attributes	Contains records (tuples)

5. Schema Diagram:

A Schema Diagram is a graphical representation of the database schema.

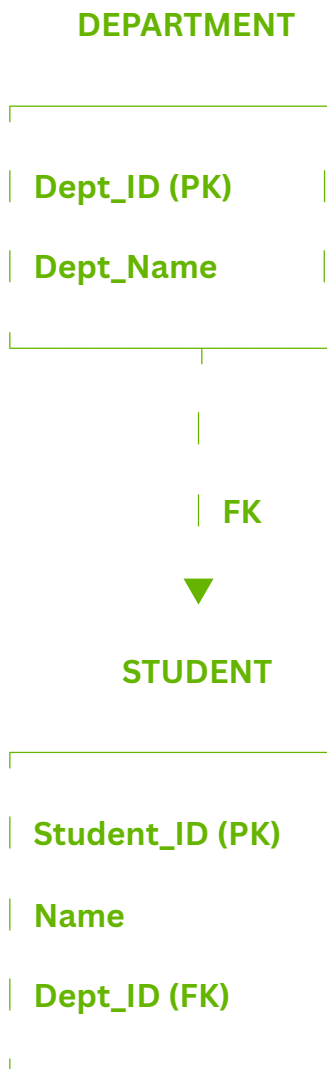
It shows:

- **Tables (Relations)**
- **Primary Keys**
- **Foreign Keys**
- **Relationships between tables**

In a schema diagram:

- **Each table is represented by a box.**
- **Primary Key attributes are underlined.**
- **Foreign Keys are shown using arrows connecting related tables.**

Example Schema Diagram-



Advantages of Logical Database Design

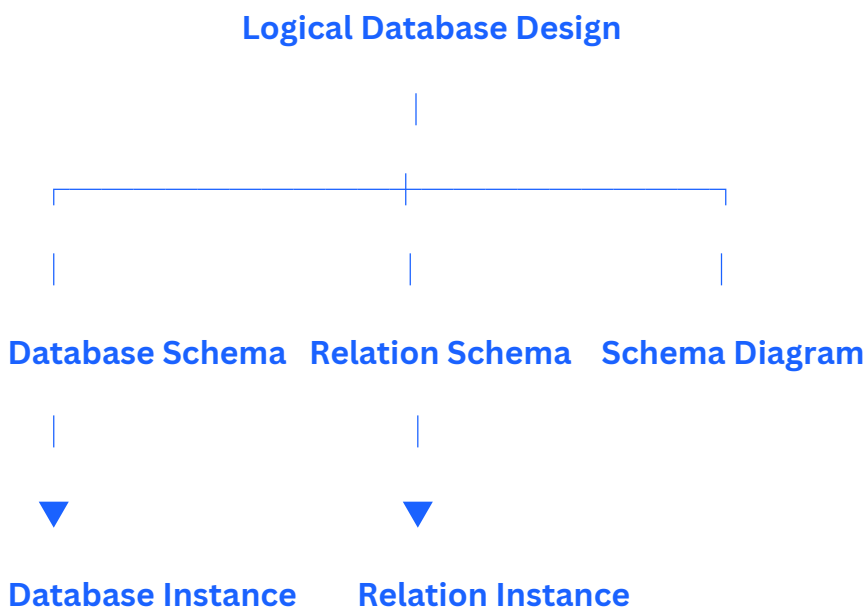
- **Organizes data systematically.**
- **Reduces redundancy.**
- **Improves data consistency.**
- **Makes database maintenance easier.**

- Defines relationships clearly.
- Supports efficient query processing.

Summary Table

Component	Description
Database Schema	Logical design of the database
Database Instance	Snapshot of the data at a given time
Relation Schema	Structure of a relation
Relation Instance	Data stored in the relation
Schema Diagram	Graphical representation of tables and relationships

Diagram:



11.Explain the Relational Model.

The Relational Model is a database model that represents data in the form of relations (tables).

Each table consists of rows (tuples) and columns (attributes). Every row represents a real-world entity, while every column represents a property (attribute) of that entity.

The relational model is the most widely used data model because of its simplicity, flexibility, and ease of use.

Characteristics of the Relational Model

- Data is organized into tables (relations).
- Each row represents a tuple (record).
- Each column represents an attribute (field).
- Every table has a unique name.
- Relationships between tables are established using keys.
- Data is stored logically rather than physically.

Terminology in the Relational Model

Term	Meaning
Relation	Table
Tuple	Row (Record)
Attribute	Column (Field)
Domain	Set of valid values of an attribute
Degree	Number of attributes (columns)
Cardinality	Number of tuples (rows)

Example-

Consider the following STUDENT relation:

Student_ID	Name	Branch	Age
101	Rahul	CSE	20
102	Ravi	ECE	21
103	Priya	IT	20

Here,

- Relation → STUDENT
- Attributes → Student_ID, Name, Branch, Age
- Tuples → Three rows
- Domain → Valid values for Student_ID, Name, Branch, and Age

Components of the Relational Model-

1. Relation (Table):

A Relation is a table that stores related data in rows and columns.

Example:

STUDENT(Student_ID, Name, Branch)

2. Attribute (Column):

An Attribute is a property or characteristic of an entity.

Example:

- Student_ID
- Name
- Branch
- Age

3. Tuple (Row):

A Tuple is a single row in a relation.

Example:

(101, Rahul, CSE, 20)

4. Domain:

A Domain is the set of permissible values for an attribute.

Example:

- Age → 18–60
- Branch → {CSE, ECE, IT, ME}

Advantages of the Relational Model-

- Simple and easy to understand.
- Reduces data redundancy.
- Maintains data integrity.
- Supports data independence.
- Easy to retrieve and manipulate data using SQL.
- Efficiently represents relationships among data.

Diagram:

RELATIONAL MODEL

DATABASE

RELATION (TABLE)

STUDENT

ID	Name	Age
101	Rahul	20
102	Ravi	21

RELATION (TABLE)

DEPARTMENT

Dept	DName
10	CSE
20	ECE

Summary Table

Component	Description
Relation	Table
Tuple	Row
Attribute	Column
Domain	Set of valid values
Degree	Number of columns
Cardinality	Number of rows

12.Differentiate between Normalization and Denormalization.

Normalization and Denormalization are two database design techniques used to organize data.

- Normalization reduces data redundancy and improves data integrity.
- Denormalization introduces controlled redundancy to improve query performance.

Normalization is generally performed first, and denormalization is applied later if performance optimization is required.

Normalization:

Normalization is the process of organizing data into smaller related tables to:

- Reduce data redundancy.
- Eliminate insertion, update, and deletion anomalies.
- Improve data consistency and integrity.

Normalization divides large tables into multiple related tables using keys.

Advantages of Normalization-

- Reduces data redundancy.
- Eliminates anomalies.
- Improves data integrity.
- Makes updates easier.
- Saves storage space.

Denormalization:

Denormalization is a database optimization technique in which redundant data is intentionally added to one or more tables to improve query performance.

It is generally applied after normalization.

Advantages of Denormalization-

- Enhances query performance.
- Reduces the number of joins.
- Makes the database more convenient to manage.
- Facilitates and accelerates report generation

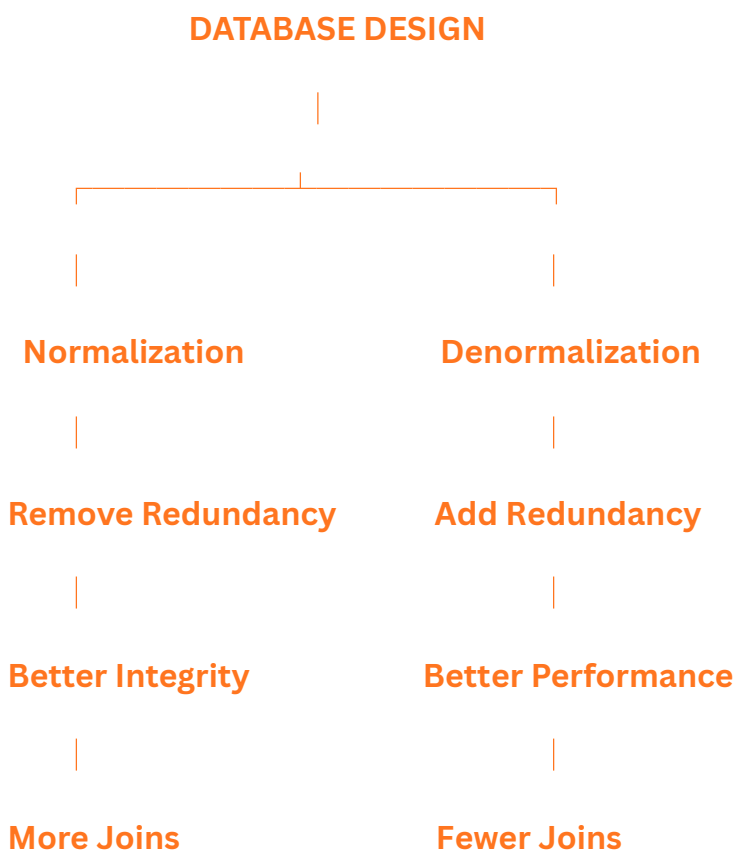
Disadvantages of Denormalization-

- Requires more storage due to redundancy.
- Makes updates and inserts more expensive.
- Increases the possibility of data inconsistency.
- Duplicate data must be updated everywhere.

Difference between Normalization and Denormalization:

Normalization	Denormalization
Removes redundant data.	Adds redundant data.
Improves data integrity.	Improves query performance.
Reduces redundancy.	Increases redundancy.
Uses optimized memory.	Requires more memory.
Suitable for frequent INSERT, UPDATE and DELETE operations.	Suitable for frequent SELECT operations and reporting.
Requires more table joins.	Reduces the number of joins.
Focuses on consistency.	Focuses on speed.
Prevents anomalies.	May introduce inconsistency if not maintained properly.

Diagram:



Applications:

Normalization-

- Banking systems
- Student databases
- Hospital management systems
- Library management systems

Denormalization-

- Data warehouses
- Reporting systems
- Business Intelligence (BI)
- Read-intensive applications



SAQs

1. Define Functional Dependency with an example.

A Functional Dependency (FD) is a relationship in which one attribute determines another attribute. It is represented as $X \rightarrow Y$, where X is the determinant and Y is the dependent attribute.

Example:

$\text{Roll_No} \rightarrow \text{Name}$

$\text{Roll_No} \rightarrow \text{Dept_Name}$

2. What is a Determinant?

A Determinant is the attribute or set of attributes on the left-hand side of a Functional Dependency that determines the value of another attribute.

Example:

$\text{Roll_No} \rightarrow \text{Name}$

Here, Roll_No is the determinant.

3. State Armstrong's Axioms.

Armstrong's Axioms are:

- Reflexivity: If $Y \subseteq X$, then $X \rightarrow Y$.
- Augmentation: If $X \rightarrow Y$, then $XZ \rightarrow YZ$.
- Transitivity: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

4. Define Trivial Functional Dependency.

A Functional Dependency is Trivial if the dependent attribute is a subset of the determinant.

Example:

$\{\text{Roll_No}, \text{Name}\} \rightarrow \text{Name}$

5. Define Non-Trivial Functional Dependency.

A Functional Dependency is Non-Trivial if the dependent attribute is not a subset of the determinant.

Example:

$\text{Roll_No} \rightarrow \text{Name}$

6. Define Partial Dependency.

A Partial Dependency occurs when a non-key attribute depends only on a part of a composite key rather than the whole key.

Example:

$(\text{Student_ID}, \text{Course_ID}) \rightarrow \text{Course_Fee}$ But,

$\text{Course_ID} \rightarrow \text{Course_Fee}$

7. Define Full Functional Dependency.

A Full Functional Dependency exists when an attribute depends on the entire key.

Example:

$\text{Roll_No} \rightarrow \text{Name}$

8. Define Transitive Dependency.

A Transitive Dependency exists when one attribute depends on another indirectly.

If

$A \rightarrow B$

$B \rightarrow C$

then

$A \rightarrow C$

9. Define Multivalued Dependency.

A Multivalued Dependency occurs when one attribute determines multiple independent values of another attribute.

Notation:

$A \twoheadrightarrow B$

10. What is Normalization?

Normalization is the process of organizing data to reduce redundancy and eliminate insertion, update, and deletion anomalies.

11. What is First Normal Form (1NF)?

A relation is in 1NF if every attribute contains only atomic (single) values.

12. What is Second Normal Form (2NF)?

A relation is in 2NF if:

- **It is in 1NF.**
- **It has No Partial Dependency.**

13. What is Third Normal Form (3NF)?

A relation is in 3NF if:

- **It is in 2NF.**
- **It has No Transitive Dependency.**

14. What is BCNF?

BCNF (Boyce-Codd Normal Form) is an advanced form of 3NF in which every determinant must be a Super Key.

15. What is Fourth Normal Form (4NF)?

A relation is in 4NF if:

- **It is in BCNF.**
- **It has no Multivalued Dependency.**

16. What is Fifth Normal Form (5NF)?

5NF or Projected Normal Form (PJNF) removes Join Dependency and ensures Lossless Join Decomposition.

17. What is Domain-Key Normal Form (DKNF)?

A relation is in DKNF if all constraints are based only on Domain Constraints and Key Constraints.

18. Define Lossless Decomposition.

Lossless Decomposition is the decomposition of a relation into smaller relations such that the original relation can be reconstructed without any loss of information.

19. Define Lossy Decomposition.

Lossy Decomposition is the decomposition of a relation in which some information is lost, and the original relation cannot be reconstructed accurately.

20. What are Insertion Anomalies?

An Insertion Anomaly occurs when data cannot be inserted because some other related data is missing.

21. What are Update Anomalies?

An Update Anomaly occurs when duplicate data is updated in one place but not everywhere, causing inconsistency.

22. What are Deletion Anomalies?

A Deletion Anomaly occurs when deleting one record unintentionally removes other important information.

23. Define Denormalization.

Denormalization is the process of adding redundant data to improve query performance. It is generally applied after normalization.

24. What are Integrity Constraints?

Integrity Constraints are rules that ensure the accuracy, validity, and consistency of data in a database.

25. List the types of Integrity Constraints.

The four types are:

- Domain Integrity
- Entity Integrity
- Referential Integrity
- Key Constraints

26. Define Domain Integrity.

Domain Integrity ensures that an attribute contains only valid values based on its data type, range, and constraints.

27. Define Entity Integrity.

Entity Integrity states that the Primary Key cannot contain NULL values.

28. Define Referential Integrity.

Referential Integrity ensures that every Foreign Key either matches a Primary Key in another table or is NULL.

29. Define Key Constraint.

A Key Constraint ensures that a key uniquely identifies each tuple in a relation.

30. What is Logical Database Design?

Logical Database Design is the logical representation of a database that defines tables, attributes, and relationships.

31. Define Database Schema.

A Database Schema is the logical design or blueprint of a database.

32. Define Relation Schema.

A Relation Schema consists of the relation name, attributes, and their domains.

33. Define Relation Instance.

A Relation Instance is the set of tuples (rows) present in a relation at a particular time.

34. What is a Schema Diagram?

A Schema Diagram is a graphical representation of tables, their attributes, primary keys, foreign keys, and relationships.

35. What is a View?

A View is a virtual table created from one or more tables using an SQL query.

36. What is WITH CHECK OPTION?

WITH CHECK OPTION ensures that every INSERT and UPDATE performed through a view satisfies the condition specified in the view definition.

37. When can a View be updated?

A view can be updated if it does not contain:

- DISTINCT
- GROUP BY
- HAVING
- Aggregate functions
- ORDER BY
- Multiple tables
- Subqueries

38. Define Super Key.

A Super Key is a set of one or more attributes that uniquely identifies a tuple.

39. Define Candidate Key.

A Candidate Key is a minimal Super Key with no redundant attributes.

40. Define Primary Key.

A Primary Key is the selected Candidate Key that uniquely identifies every record and cannot contain NULL values.

41. Define Alternate Key.

An Alternate Key is a Candidate Key that is not selected as the Primary Key.

42. Define Foreign Key.

A Foreign Key is an attribute that references the Primary Key of another table, maintaining referential integrity.

43. Define Composite Key.

A Composite Key is a key formed by combining two or more attributes to uniquely identify a record.

44. Define Surrogate Key.

A Surrogate Key is a system-generated unique identifier that is not derived from application data.

45. Define Unique Key.

A Unique Key ensures that all values in an attribute are unique. Unlike the Primary Key, it may allow NULL values.

1. Querying Relational Data

Querying Relational Data is the process of retrieving required data from one or more relational database tables.

A Query is a request for data or information from a database. Queries are generally written using Structured Query Language (SQL).

The retrieved data may be displayed as:

- Tables
- Reports
- Graphs
- Statistical analysis

SQL is the most widely used query language for performing simple as well as complex database queries.

Example:

```
SELECT Name, Age FROM STUDENT WHERE Age > 18;
```

This query retrieves the names and ages of students whose age is greater than 18.

2. Dependency Preservation

Dependency Preservation means that after decomposing a relation into two or more relations, all the Functional Dependencies of the original relation can still be enforced without joining the decomposed relations.

No Functional Dependency should be lost after decomposition.

Need for Dependency Preservation

- Preserves all Functional Dependencies.
- Makes constraint checking easier.
- Avoids unnecessary joins.
- Improves database efficiency.

Example:

Suppose

$R(A, B, C)$

$A \rightarrow B$

$B \rightarrow C$

After decomposition:

$R_1(A, B)$

R2(B, C)

Both Functional Dependencies

$A \rightarrow B$

$B \rightarrow C$ can still be enforced.

Hence,

Dependency Preservation is achieved.

Key Point

A good decomposition should satisfy:

- Lossless Join
- Dependency Preservation

3. User-Level and System-Level Privileges

A Privilege is a permission granted to a user to perform specific operations on database objects.

Privileges are of two types:

1. User-Level Privileges
2. System-Level Privileges

1. User-Level Privileges (Object Privileges)

These privileges allow a user to perform operations on specific database objects like tables or views.

Examples

- SELECT
- INSERT
- UPDATE
- DELETE
- REFERENCES
- ALTER

Example:

```
GRANT SELECT ON STUDENT TO USER1;
```

This allows USER1 to read the STUDENT table.

2. System-Level Privileges

These privileges allow users to perform database administration tasks.

Examples

- CREATE TABLE
- CREATE VIEW
- CREATE USER
- DROP TABLE
- ALTER DATABASE

Example:

GRANT CREATE TABLE TO USER1;

This allows USER1 to create tables in the database.

Difference between User-Level and System-Level Privileges

User-Level Privileges	System-Level Privileges
Apply to specific database objects	Apply to the entire database
Granted on tables, views, etc.	Granted for administrative operations
Example: SELECT, INSERT, UPDATE	Example: CREATE TABLE, CREATE USER
Used by normal users	Used mainly by DBAs

Advantages

- Improves database security.
- Controls user access.
- Prevents unauthorized operations.
- Protects sensitive data.